已同意录用，作为第27届 Information Security Conference会议论文（CCF C类）等待见刊。

由于刊物近期才会发表，故没有论文期刊封面

MVDetector: Malicious vehicles detection under Sybil attacks in VANETs

Yiming Wei[1], Weiye Qi[1], Zechuan Li[1], Yufan Han[1], and Yingxu Lai[1,2*]

[1] Beijing University of Technology, Beijing, China
{weiyiming,qiweiye,lizechuan,hyf}@emails.bjut.edu.cn
[2] Science and Technology on Information Assurance Laboratory, Beijing, China
laiyingxu@bjut.edu.cn

Abstract. A Sybil attack involves a Malicious vehicle node stealing fake identities to continuously generate fake vehicles on the road, creating an illusion of congestion and interfering with the normal traffic flow of legitimate vehicles. In the current traffic environment, vehicles cannot perform real-time authentication, allowing highly stealthy Malicious vehicle nodes to continue attacking and significantly impact traffic. Given the rapidly changing network topology in vehicular networks, high precision and speed are required for attack detection methods. This paper proposes a three-class classification method for Sybil vehicles, Malicious vehicles, and Normal vehicles based on BSM packets. This method utilizes multiple features and employs a sliding window with the Random Forest algorithm for classification. Compared with deep learning methods, this method has the advantages of strong interpretability and fast detection speed. Experiments demonstrate that this method achieves fast detection speeds and high accuracy. With a window size of 2, the method achieves precision and recall both greater than 94%.

Keywords: ITS · VANETs · Sybil detection · Malicious detection

1 Introduction

With the rapid development of information technology and connected autonomous vehicle (CAV) technology, Vehicular Ad Hoc Networks (VANETs) have become a significant area of research. VANETs encompass both vehicle-to-vehicle (V_2V) and vehicle-to-infrastructure (V_2I) communication streams [16]. Vehicular Ad Hoc Networks (VANETs) [14] are an essential component of Intelligent Transportation Systems (ITS) [6,11], contributing to improved traffic conditions and enhanced driving safety in VANETs, vehicles need to periodically broadcast Basic Safety Messages (BSMs) that contain critical vehicle status information as well as event-driven notifications. This ensures that their status information, such as location and speed, can be acquired by other nodes within the VANET, thereby ensuring road safety and improving traffic flow [12].

* Corresponding author.

Vehicles communicate using wireless broadcast, which makes it easy for attackers to forge or alter certain fields of Basic Safety Message (BSM) packets, posing a significant threat to traffic safety. Additionally, due to the high mobility of vehicles, Vehicular Ad Hoc Networks (VANETs) are characterized by rapid changes in their composition and structure. To effectively protect privacy information in the network, vehicles communicate using pseudonyms [18]. Consequently, attackers can easily steal or forge pseudonyms to conduct Sybil attacks, making it very difficult to detect the attackers and verify their true identities.

A Sybil attack refers to a scenario in a peer-to-peer system where a defective or malicious entity can present multiple identities, thus influencing a significant portion of the system [5]. In Vehicular Ad Hoc Networks (VANETs), Sybil attacks manifest in the following form: a Malicious vehicle (malicious entity) illegally acquires multiple pseudonyms, creating multiple fake vehicle entities (Sybil vehicles). This disrupts the perception of normal entities within the VANET regarding road conditions, for instance, creating a false impression of road congestion, thereby affecting the autonomous driving of legitimate vehicles. Figure 1 illustrates an example of a Sybil attack. In Fig. 1(a), both Normal and Malicious vehicles are driving forward. In Fig. 1(b), the Malicious vehicle sends multiple fake Basic Safety Message (BSM) packets to the Normal vehicle. In Fig. 1(c), the fake Sybil vehicles are generated, creating an illusion of traffic congestion for the Normal vehicle, leading it to take the right exit. The Normal vehicle, deceived by the false congestion, takes the right exit, resulting in a successful attack.

The use of pseudonyms and the rapid changing characteristic of Vehicular Ad Hoc Networks (VANETs) make real-time authentication of vehicle nodes very challenging [2]. Consequently, a Malicious vehicle can easily maintain multiple fake identities to carry out sustained attacks. Moreover, the Malicious vehicle can fabricate the Basic Safety Message (BSM) packets of its virtual vehicles, such as constructing a series of BSM packets with speed, direction, and other information that make their behavior resemble that of Normal vehicles, making the virtual vehicles difficult to detect. Additionally, since the Malicious vehicle is a real entity and itself a genuine vehicle, identifying the Malicious vehicle becomes even more difficult.

There are existing methods that use machine learning (ML) or deep learning (DL) for binary classification of BSM packets to distinguish between virtual and real vehicles [9]. However, these methods cannot identify Malicious vehicles, and deep learning approaches lack interpretability. As a result, even if virtual vehicles are detected and deemed untrustworthy, Malicious vehicles can continue to forge virtual vehicles and maintain the attack. Therefore, identifying Malicious vehicle nodes is the fundamental problem that needs to be addressed.

Therefore, we propose a machine learning (ML) based method that performs three-class classification on BSM packets, allowing for the simultaneous identification of Sybil vehicles, Normal vehicles, and Malicious vehicles. This method addresses the limitation of existing approaches that use only machine learning or deep learning techniques with BSM packets, which are unable to distinguish between Normal and Malicious vehicles.

Our Contributions: **(a)** We introduced new features for three-class vehicle classification and proposed a machine learning-based method that requires only BSM packets. This approach addresses the challenge of distinguishing Sybil vehicles, Normal vehicles, and Malicious vehicles, particularly the difficulty in detecting Malicious vehicles.
(b) The accuracy of our method's three-class classification is minimally affected by the number of received BSM packets, maintaining high accuracy even with a small number of BSM packets.
(c) Existing methods have achieved high accuracy in the binary classification problem of detecting Sybil vehicles. However, our method not only accomplishes the three-class classification of Sybil vehicles, Normal vehicles, and attacking vehicles but also maintains high accuracy, with precision and recall both exceeding 94%.
Organization: The structure of the following sections is as follows. Section 2 reviews related work on Sybil attack detection. Section 3 describes the attack scenarios and attack model. Our proposed method for addressing this attack is detailed in Section 4. Section 5 presents the experimental design and analyzes the experimental results. Finally, Section 6 provides a summary of the paper.

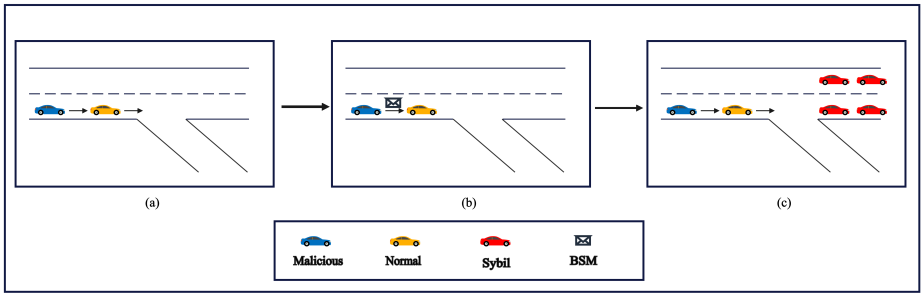


Fig. 1. Schematic of a Sybil attack.

2 Related Work

There has been considerable research into intrusion detection and attack tracing for Sybil attacks within vehicular networks, as shown in Table 1. Intrusion detection methods for Sybil attacks typically include trust-based approaches, which assess the trustworthiness of nodes based on historical interaction behaviors. Nodes exhibiting unreliable or abnormal behavior can be suspected of being Sybil nodes, necessitating frequent information exchanges to establish a trust model. Location verification methods are crucial in VANETs where accurate and credible traffic information depends on the correctness of node location data; these methods verify whether the reported locations of nodes correspond

to their actual positions. Behavior-based methods analyze the patterns and trajectories of vehicle movements, identifying potential Sybil nodes if a node moves too quickly or appears in implausible locations. Machine learning-based methods leverage anomaly detection algorithms to analyze a node's various characteristics and behavioral patterns to identify anomalies.

Kumar et al. [13] proposed a trust-based method that calculates and adjusts the trust values of nodes based on their historical interactions and real-time behavior. Nodes monitor the behavior of their neighbors and exchange trust-related information, dynamically adjusting trust scores by evaluating factors such as message forwarding behavior, message consistency, and response times. This method has shown improvements in detection efficiency and accuracy compared to previous trust-based approaches. However, it requires a substantial amount of historical data, resulting in less effective initial deployment; moreover, in highly dynamic environments, real-time calculation of trust values may introduce delays.

Mohamed Baza et al. [3] proposed a method based on continuous location proofs, where vehicle nodes submit solutions to Proof-of-Work (PoW) challenges to roadside units (RSUs) in order to obtain verification of their locations. Multiple location proofs are used to form a trajectory, which must be provided by the vehicle when reporting road events for verification by the management center. By integrating location verification with PoW, this approach enhances the security of VANETs against Sybil attacks. However, this method relies on the deployment of RSUs, which could increase infrastructure costs.

Bo Yu et al. [15] proposed a method based on vehicle behavior patterns, which involves monitoring characteristics such as speed, direction, and route consistency to detect Sybil nodes. Using statistical analysis and pattern recognition techniques, the method identifies anomalies in behavior patterns and trajectories. This approach leverages natural vehicle behaviors for Sybil node detection without the need for additional infrastructure, providing an active method of anomaly detection. However, it relies on predefined behavior patterns, which may not cover all practical situations; if attackers mimic legitimate behaviors, the effectiveness of the detection might be reduced.

Alladi et al. [1] applied deep learning to intrusion detection by collecting various feature data from the network, such as vehicle speed and direction, and training multiple models for performance comparison. They proposed single-stage and dual-stage classifiers that achieve high accuracy in detecting Sybil attacks with low resource consumption. However, this method does not enable the identification of the source of the attack, and deep learning methods have poor interpretability.

Kamel et al. [8] have proposed and evaluated several methods in their research, including threshold-based detection, Non-Cooperative Trust-Based (N-CTB), Cooperative Trust-Based (CTB), and Machine Learning (ML)-based approaches. Through experimental testing and method assessment, the ML-based approach slightly outperformed deterministic algorithms and achieved a high

level of accuracy. However, there is still room for improvement in accuracy, and the method has significant limitations in tracing the source of attacks.

Zhang et al. [17] proposed a location-based method utilizing BSM packets, which capitalizes on the low-cost feature of BSM packets having a unique source of transmission. By comparing the transmission distance calculated from the location information in the BSM packets with the distance derived from the time information and spatio-temporal functions, the method achieves detection and tracing of Sybil attacks. However, this approach may face challenges when there is an excessive volume of packets and suboptimal signal conditions. Additionally, the spatio-temporal functions relied upon for packet transmission are influenced by various factors in reality, making them more challenging to accurately model.

Table 1. Comparison of related work.

References	Contributions	Advantages	Disadvantages
Kumar et al. [13]	An effective trust-based assessment method is proposed.	• High efficiency • High accuracy	• Requires historical data; initial deployment effectiveness is poor. • Real-time calculation of trust values introduces delays.
Baza et al. [3]	A method based on continuous location proofs is proposed.	• Enhanced the security of VANETs against Sybil attacks.	• Over-reliance on RSU deployment increases infrastructure costs.
BO et al. [15]	A method based on vehicle behavior patterns is proposed.	• No additional infrastructure required. • Active anomaly detection method.	• Unable to cover all practical situations. • Difficult to detect imitative behaviors.
Alladi et al. [1]	A two-stage machine learning classifier based on BSM packets is proposed.	• Uses fewer resources. • High accuracy	• Unable to identify the source of the attack.
Joseph et al. [8]	Several methods are proposed for evaluation, with machine learning demonstrating the highest accuracy.	• Effectively detects Sybil attacks. • Various BSM-based methods.	• Unable to identify the source of the attack.
Zhang et al. [17]	A method is proposed that utilizes BSM packets based on vehicle location.	• Achieves attack tracing.	• A single spatio-temporal function cannot accommodate multiple scenarios.

To overcome some of the weaknesses of existing methods, we propose a machine learning-based detection scheme that requires only the extraction of features from Basic Safety Message (BSM) packets. This detection scheme demands fewer packets, provides rapid judgments, and uses minimal resources. Each vehicle node possesses detection capabilities, allowing for decentralized detection.

3 Attack Model

We propose a three-class classification method for Sybil vehicles, Normal vehicles, and Malicious vehicles, utilizing vehicle movement information transmitted in BSM packets as features and employing Random Forest for classification. Therefore, in this section, we will introduce the format of BSM packets used for detection and the attacker model.

3.1 BSM Packet Format

The BSM packets transmitted between vehicles contain the following information: sendTime, senderPseudo, messageID, pos, spd, acl, hed. The meanings of these fields are presented in the Table 2.

Table 2. BSM packet field definitions.

Field	Meaning
sendTime	Time of occurrence of BSM packet
senderPseudo	Pseudonym of the sending vehicle
messageID	Identification number of the BSM packet
pos	Position of the vehicle
spd	Speed of the vehicle
acl	Acceleration of the vehicle
hed	Heading direction of the vehicle

3.2 Attack Model

Sybil attacks are initiated by Malicious vehicles, executed through Sybil vehicles. In launching a Sybil attack, Malicious vehicles generate multiple Sybil vehicles, causing nearby vehicles to incorrectly perceive the surrounding road conditions as congested, disrupting normal traffic order. As depicted in Fig. 2(a), Malicious vehicle V_0 broadcasts false BSM packets to forward vehicle V_1 at different time instances. V_1 captures the contents of BSM packets sent at three different time instances, which include sendTime, senderPseudo, messageID, pos, spd, acl, and hed. Upon receiving false BSM packets, V_1 generates Sybil vehicle nodes, as illustrated in Fig. 2(b). In the diagram, V_2, V_3, and V_4 are Sybil vehicle nodes

generated by Malicious vehicle node V_0. The generated Sybil vehicle nodes block the driving direction of Normal vehicle node V_1, leading to V_1 misjudging the current road condition as congested. Thus, the Malicious vehicle successfully executes the Sybil attack.

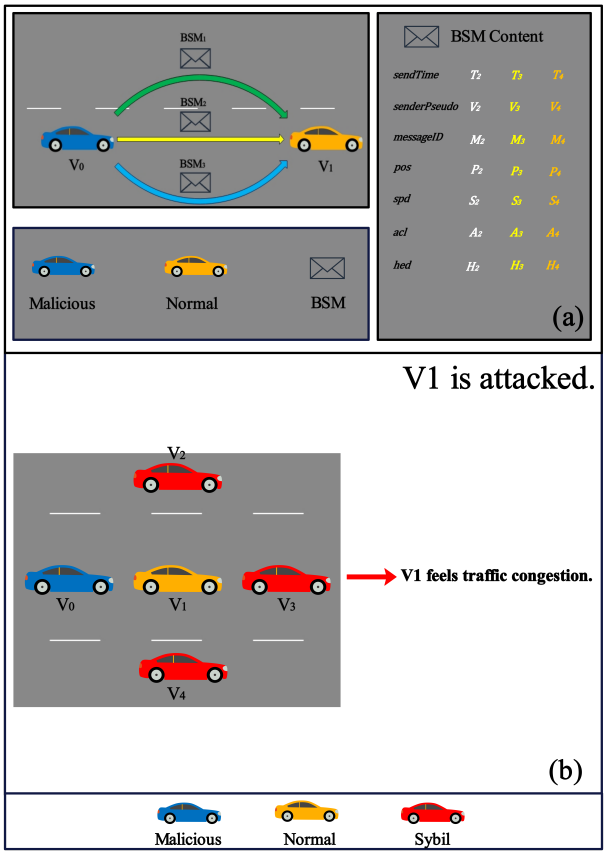


Fig. 2. Schematic of a grid Sybil attack.

4 Method

This section introduces the three-class classification method for Sybil vehicles, Normal vehicles, and Malicious vehicles based on BSM packets. The method consists of three parts: data processing, sliding window creation, and the Random Forest model, as shown in Figure 3.

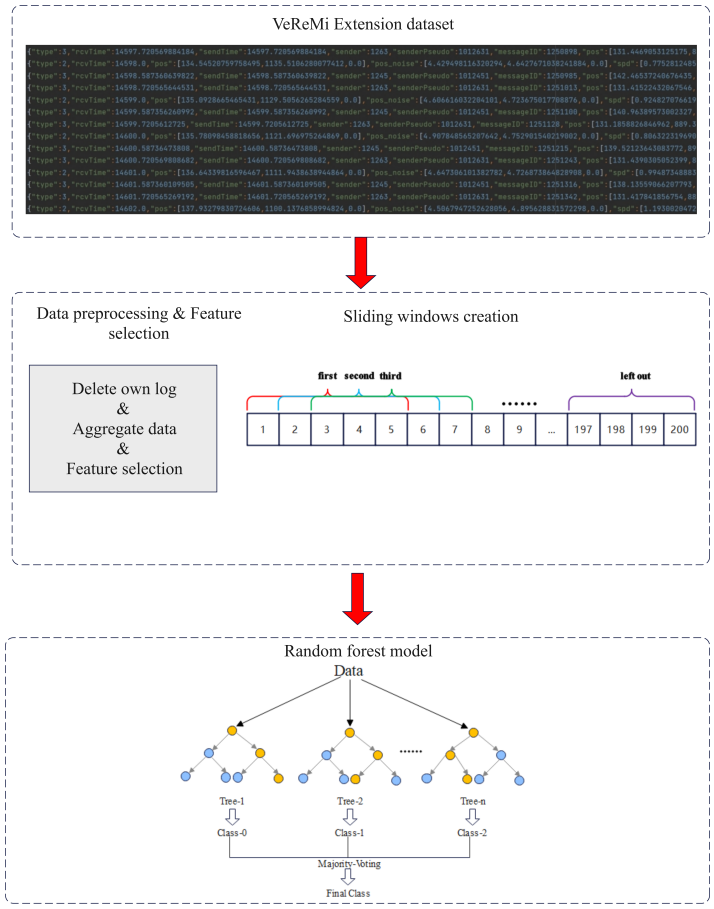


Fig. 3. A three-classification system for vehicles in Sybil attacks.

4.1 Data Processing

Dataset Structure: The VeReMi extended dataset is a public dataset obtained from the F2MD (Falsified Driving Misbehavior Detection) open-source framework [10,7] in the LuSTNanoScenario simulation scenario. The LuSTNanoScenario is part of the LuST scenario, which encompasses most real-world traffic situations, such as city streets, highways, and roundabouts. The VeReMi extended dataset consists of 24 simulations, each corresponding to one hour from 0 to 23. Each simulation includes the raw tracking logs of every vehicle traversing the network within that specific hour, the data received from other vehicles, and a ground truth (GT) file. Each message comprises fields such as sendTime, senderID, senderPseudo, messageID, and location-based information including position, speed, acceleration, and heading, along with noise in each value. The

quantities of Normal, Sybil, and Attack Source data we used are shown in the Table 3.

Table 3. Three types of vehicle data.

Data Type	Quantity
Normal	40697
Sybil	40649
Malicious	40904

Data Preprocessing and Feature Selection: We removed the individual log files of each vehicle and aggregated the information received from other vehicles based on different values of senderPseudo, facilitating subsequent training. We selected three major categories of motion information as features: acceleration, speed, and direction. The specific features under each category are listed in the Table 4.

Table 4. Three types of vehicle data.

Category	Features
Acceleration	pos_ratio_x: Ratio of positive to negative acceleration in the X direction pos_ratio_y: Ratio of positive to negative acceleration in the Y direction Dispersion: Sum of variances of acceleration in the X and Y directions var_x: Variance of acceleration values in the X direction var_y: Variance of acceleration values in the Y direction mean_acl_x: Mean acceleration value in the X direction mean_acl_y: Mean acceleration value in the Y direction
Speed	mean_spd: Mean speed value var_spd: Variance of speed values
Direction	mean_dir_change: Mean value of direction change (in degrees) max_dir_change: Maximum value of direction change (in degrees) var_dir_change: Variance of direction change (in degrees)

4.2 Sliding Window Creation

Based on the previously processed data, features are extracted using a sliding window approach. Starting from the first data entry, the features listed in the Table 4 are extracted according to the window size, with each extraction moving by a specified step size. If the number of data entries is insufficient for the window size during sliding, those data entries will be ignored. For example, as shown in Figure 4, with a total of 200 data entries, a window size of 5, and a stride of

1, the extracted feature data ranges sequentially from 1-5, 2-6, 3-7, ..., 196-200, 197-200, 198-200, 199-200, etc. Any data that does not meet the window size is ignored.

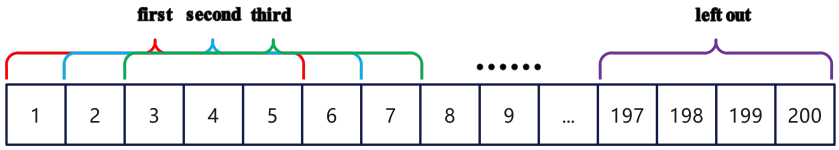


Fig. 4. Sliding window diagram.

4.3 Classifier

Random Forest Model: Random forests are a combination of tree predictors such that each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest. The generalization error for forests converges as to a limit as the number of trees in the forest becomes large [4].

Random Forest Model Parameters: The model parameters are set to class weight = 'balanced' to address the issue of class imbalance, ensuring that the weight of each class is automatically adjusted according to its sample size. The number of trees in the forest (`n_estimators`) is set to 100, meaning the forest contains 100 decision trees. The criterion for splitting nodes is Gini impurity, which is used to evaluate the quality of splits.

In our study, we used features extracted from sliding windows to train the model. By learning the features extracted from each window, the model can effectively distinguish between the three types of vehicle data. Experimental validation shows that, based on our proposed new features, the Random Forest model outperforms other models in the classification task, achieving the best classification results.

5 Experiment and result

In this section, we conducted experiments on Sybil attack detection within Vehicular Ad Hoc Networks (VANETs). We implemented a three-class classification method to distinguish between Normal vehicles, Sybil vehicles, and Malicious vehicles, and evaluated the performance of various machine learning algorithms. We focused on analyzing the performance of Random Forest, Decision Tree, Multilayer Perceptron (MLP), Gradient Boosting, and Naive Bayes algorithms under our proposed new features for this task. Additionally, we explored the performance variations of these algorithms under different window sizes to determine the optimal window settings, ensuring the efficiency and accuracy of the detection system. Finally, we also compared our method with existing binary classification approaches for detecting Sybil vehicles.

This experiment is based on the F2MD [10,7] (Falsified Driving Misbehavior Detection) open-source framework for detecting improper driving behavior.

5.1 Performance Evaluation of New Features in ML Algorithms

Data preprocessing and feature selection are discussed in Section 4.1.

In this study, the primary purpose of data preprocessing is to provide a clean and effective training dataset for the model. By organizing and transforming the original BSM data, we ensured the quality of the input data to more accurately simulate and identify Sybil attacks.

We experimented with our newly proposed features using various machine learning algorithms to evaluate their performance in the three-class classification of Normal vehicles, Sybil vehicles, and Malicious vehicles. We assessed the performance of several algorithms for this task, including Decision Tree, Random Forest, Multilayer Perceptron (MLP), Gradient Boosting, and Naive Bayesian. Figure 5 and Table 5 display the performance data for these machine learning algorithms, including Decision Trees, Random Forests, MLP, Gradient Boosting, and Naive Bayesian. X_2 represents the performance of X algorithm when experiments are conducted with a window size of 2.

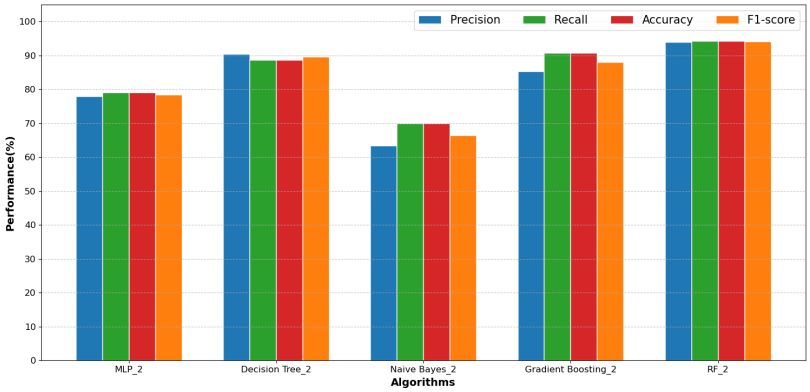


Fig. 5. Performance of New Features in Various Classifiers.

In this study, we set the window size to 2 and the stride to 1 to balance the model's detection sensitivity and real-time performance. A smaller window size allows the model to respond more quickly to sudden changes in behavior, which is crucial for real-time identification of potential Sybil attacks in vehicular network environments. Additionally, a stride of 1 ensures that every data point in the data stream is fully utilized, enhancing the model's ability to capture the dynamics of vehicle behavior.

From Table 5, it is evident that the new features we proposed perform well across all classifiers. Notably, the Random Forest algorithm achieved a high level

Table 5. Performance of New Features in Various Classifiers.

Evaluation Metrics	Detection Solution				
	Decision Tree_2	RF_2	MLP_2	Gradient Boosting_2	Naive Bayesian_2
Precision	90.5%	94.0%	77.9%	85.4%	63.4%
Recall	88.7%	94.3%	79.1%	90.7%	70.0%
Accuracy	88.7%	94.3%	79.1%	90.7%	70.0%
F1-score	89.6%	94.2%	78.5%	88.0%	66.5%

over 94% in precision, accuracy, and F1 score, significantly outperforming other algorithms. Gradient Boosting and Decision Tree algorithms also demonstrated remarkable performance, achieving over 85% in precision, accuracy, and F1 score, ranking just behind Random Forest.

5.2 Performance of Random Forest Under Different Windows

Experiments were conducted on the Random Forest algorithm with varying window sizes while maintaining a stride of 1 to achieve optimal performance. Choosing a fixed stride ensures complete coverage of data, allowing the model to continuously and comprehensively analyze every piece of data transmitted in the vehicular network environment. Figure 6 and Table 6 show the performance of the Random Forest algorithm at a fixed stride of 1 with different window sizes. RF_i represents the performance of the Random Forest algorithm when experiments are conducted with a window size of i.

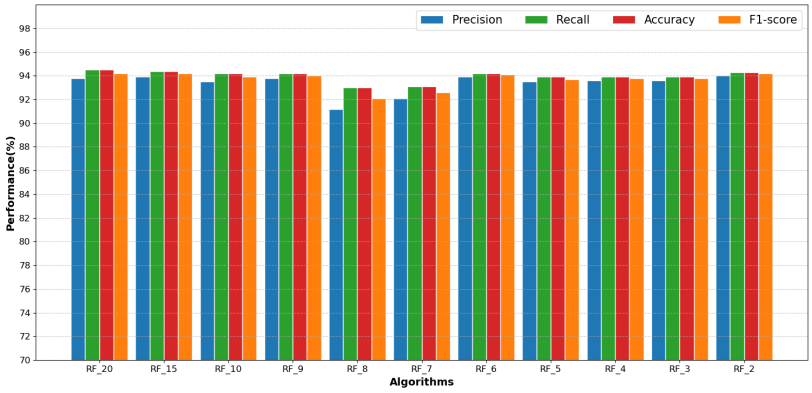


Fig. 6. Evaluation Metrics of Random Forest at Different Window Sizes.

Table 6 shows that the performance of the Random Forest algorithm varies little across different window sizes. While increasing the window size enhances

Table 6. Evaluation Metrics of Random Forest at Different Window Sizes.

Evaluation Metrics	Detection Solution					
	RF_20	RF_15	RF_10	RF_9	RF_8	RF_7
Precision	93.8%	93.9%	93.5%	93.8%	91.2%	92.1%
Recall	94.5%	94.4%	94.2%	94.2%	93.0%	93.1%
Accuracy	94.5%	94.4%	94.2%	94.2%	93.0%	93.1%
F1-score	94.2%	94.2%	93.9%	94.0%	92.1%	92.6%

Evaluation Metrics	Detection Solution				
	RF_6	RF_5	RF_4	RF_3	RF_2
Precision	93.9%	93.5%	93.6%	93.6%	94.0%
Recall	94.2%	93.9%	93.9%	93.9%	94.3%
Accuracy	94.2%	93.9%	93.9%	93.9%	94.3%
F1-score	94.1%	93.7%	93.8%	93.8%	94.2%

the model's data processing capability by providing more information for decision-making, thus improving its robustness, it also implies increased latency. In Vehicular Ad Hoc Networks (VANETs), where Basic Safety Messages (BSM) are received once every second, larger window sizes result in prolonged detection response times. For example, if the window size is 20, it takes 20 seconds to make the first judgment. During this time, a Malicious vehicle could likely have completed its attack, resulting in detection failure.

Therefore, we focus more on experimental data with smaller window sizes. As shown in Table 6, when the window size is 2, the Random Forest algorithm achieves precision of 94.0%, recall of 94.3%, accuracy of 94.3%, and F1 score of 94.2%. A window size of 2 allows for the rapid and effective identification and classification of Normal vehicles, Sybil vehicles, and Malicious vehicles in Vehicular Ad Hoc Networks (VANETs), offering advantages in both detection accuracy and response time. This demonstrates significant practical relevance and application value.

5.3 Comparison with existing methods

Joseph et al. [8] proposed Sybil detection methods including threshold-based detection, Non-Cooperative Trust-Based (N-CTB), Cooperative Trust-Based (CTB), MLP, and XGBoost. Figure 7 and Table 7 display the performance data of these methods. Although these approaches already exhibit good performance, they only perform detection of Sybil vehicles and do not implement a three-class classification.

Table 7 shows that the XGBoost algorithm performs excellently across all evaluation metrics, achieving precision and recall of 98.2%. The MLP algorithm follows closely behind, also showing impressive performance in precision, recall, and F1 score, with values of 95.1%, 93.7%, and 94.4% respectively. This demonstrates its high accuracy and reliability in the binary classification task of

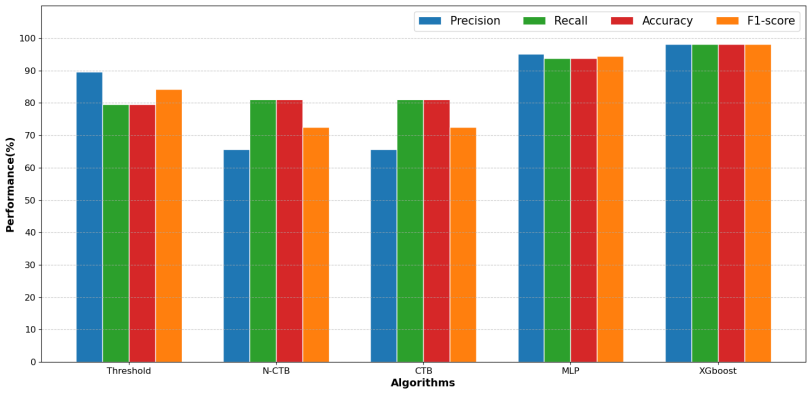


Fig. 7. Performance of existing methods.

Table 7. Performance of existing methods.

Evaluation Metrics	Detection Solution				
	Threshold	N-CTB	CTB	MLP	XGBoost
Precision	89.6%	65.7%	65.7%	95.1%	98.2%
Recall	79.5%	81.0%	81.0%	93.7%	98.2%
Accuracy	79.5%	81.0%	81.0%	93.7%	98.2%
F1-score	84.3%	72.6%	72.6%	94.4%	98.2%

Sybil detection. Although XGBoost performs excellently in the binary classification task of detecting Sybil vehicles, detection in Vehicular Ad Hoc Networks (VANETs) involves more than distinguishing between physical vehicles and Sybil vehicles. It is also crucial to differentiate between normal physical vehicles and malicious physical vehicles.

Our three-class method, by introducing a third category (Malicious Vehicles), enhances the overall detection capabilities of the system. Particularly with a window size of 2, our method achieves high levels in precision, recall, accuracy, and F1 score.

6 Conclusion

In this paper, we propose a three-class classification method for Sybil vehicles, Malicious vehicles, and Normal vehicles based on Basic Safety Message (BSM) packets. We introduced multiple new features and utilized a sliding window with the Random Forest algorithm for three-class classification. As the detection accuracy of our method is minimally affected by the size of the sliding window, it maintains high precision even with smaller windows, resulting in fast detection speeds. Experiments show that our method performs comparably to existing methods in Sybil detection, but goes a step further by successfully classifying

Malicious vehicles and Normal vehicles. Experimental results indicate that with a window size of 2, our method achieves precision and recall both $> 94\%$.

Disclosure of Interests. The authors declare that there are no competing interests relevant to this study.

References

1. Alladi, T., Kohli, V., Chamola, V., Yu, F.R.: A deep learning based misbehavior classification scheme for intrusion detection in cooperative intelligent transportation systems. Digital Communications and Networks **9**(5), 1113–1122 (2023)
2. Awais, S.M., Yucheng, W., Mahmood, K., Akram, M.W., Hussain, S., Das, A.K., Park, Y.: Puf-based privacy-preserving simultaneous authentication among multiple vehicles in vanet. IEEE Transactions on Vehicular Technology (2023)
3. Baza, M., Nabil, M., Mahmoud, M.M., Bewermeier, N., Fidan, K., Alasmary, W., Abdallah, M.: Detecting sybil attacks using proofs of work and location in vanets. IEEE Transactions on Dependable and Secure Computing **19**(1), 39–53 (2020)
4. Breiman, L.: Random forests. Machine learning **45**, 5–32 (2001)
5. Douceur, J.R.: The sybil attack. In: International workshop on peer-to-peer systems. pp. 251–260. Springer (2002)
6. Gaurav, A., Psannis, K., Perakovic, D.: Security of cloud-based medical internet of things (miots): A survey. International Journal of Software Science and Computational Intelligence **14**, 1–16 (01 2022). https://doi.org/10.4018/IJSSCI.285593
7. Kamel, J., Ansari, M.R., Petit, J., Kaiser, A., Jemaa, I.B., Urien, P.: Simulation framework for misbehavior detection in vehicular networks. IEEE transactions on vehicular technology **69**(6), 6631–6643 (2020)
8. Kamel, J., Jemaa, I.B., Kaiser, A., Cantat, L., Urien, P.: Misbehavior detection in c-its: A comparative approach of local detection mechanisms. In: 2019 IEEE Vehicular Networking Conference (VNC). pp. 1–8. IEEE (2019)
9. Kamel, J., Jemaa, I.B., Kaiser, A., Urien, P.: Misbehavior reporting protocol for c-its. 2018 IEEE Vehicular Networking Conference (VNC) pp. 1–4 (2018), https://api.semanticscholar.org/CorpusID:59553848
10. Kamel, J., Wolf, M., Van Der Hei, R.W., Kaiser, A., Urien, P., Kargl, F.: Veremi extension: A dataset for comparable evaluation of misbehavior detection in vanets. In: ICC 2020-2020 IEEE International Conference on Communications (ICC). pp. 1–6. IEEE (2020)
11. Kumar, R., Singh, A.: A two-phase load balancing algorithm for cloud environment. Int. J. Softw. Sci. Comput. Intell. **13**(1), 38–55 (apr 2024). https://doi.org/10.4018/IJSSCI.2021010103, https://doi.org/10.4018/IJSSCI.2021010103
12. Liu, X., Amour, B.S., Jaekel, A.: A q-learning based adaptive congestion control for v2v communication in vanet. In: 2022 International Wireless Communications and Mobile Computing (IWCMC). pp. 847–852 (2022). https://doi.org/10.1109/IWCMC55113.2022.9824995
13. V, Kumar, S., R, P.R.B.D.: Sybil attack detection in vehicular ad-hoc networks using direct trust calculation. International Journal of Innovative Technology and Exploring Engineering **9**(10), 67–73 (2020)
14. Xia, Z., Wu, J., Wu, L., Chen, Y., Yang, J., Yu, P.S.: A comprehensive survey of the key technologies and challenges surrounding vehicular ad hoc networks. ACM Trans. Intell. Syst. Technol. **12**(4) (jun 2021). https://doi.org/10.1145/3451984, https://doi.org/10.1145/3451984

15. Yu, B., Xu, C.Z., Xiao, B.: Detecting sybil attacks in vanets. Journal of Parallel and Distributed Computing **73**(6), 746–756 (2013)
16. Yu, H., Liu, R., Li, Z., Ren, Y., Jiang, H.: An rsu deployment strategy based on traffic demand in vehicular ad hoc networks (vanets). IEEE Internet of Things Journal **9**(9), 6496–6505 (2022). https://doi.org/10.1109/JIOT.2021.3111048
17. Zhang, Z., Lai, Y., Chen, Y., Wei, J., Wang, Y.: Detection method to eliminate sybil attacks in vehicular ad-hoc networks. Ad Hoc Networks **141**, 103092 (2023)
18. Zoccoli, G.G., Stabili, D., Marchetti, M.: Are vanets pseudonyms effective? an experimental evaluation of pseudonym tracking in adversarial scenario. In: 2023 IEEE 98th Vehicular Technology Conference (VTC2023-Fall). pp. 1–6. IEEE (2023)